Lab 10 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — 40 GB in `userdata/`

Antwoord. Dat is het **logbestand** van de container (`ctr.log`, driver `k8s-file`): een praatzieke applicatie die al weken draait, zonder rotatie. `podman system df` telt de images, de writable layers en de volumes, niet de logs: in zijn ogen zijn ze "terug te winnen" noch "te verwijderen". Onmiddellijke correctie: de container opnieuw aanmaken (de logs gaan met hem weg) of het bestand afkappen (`: > ctr.log`, aanvaardbaar in nood). Duurzame correctie: de grootte begrenzen (`--log-opt max-size=10m`, of `log_size_max` in `containers.conf`), de logs naar een centraal platform sturen, en het logniveau van de applicatie verlagen.

Waarom. De engine vangt stdout op in een bestand dat standaard geen limiet heeft; alleen een rotatie of een maximumgrootte houdt het tegen.

Nuance. Met de driver `journald` (standaard op een Ubuntu met systemd) groeit het systeemjournaal, begrensd door zijn eigen configuratie (`SystemMaxUse`).

Voorbeeld.

```
podman inspect api --format '{{.HostConfig.LogConfig.Type}} {{.HostConfig.LogConfig.Path}}'
du -sh $(podman inspect api --format '{{.HostConfig.LogConfig.Path}}')
```

Vraag 2 — `Exited (137)` zonder `OutOfMemoryError`

Antwoord. Met `MaxRAMPercentage=90` kan de Java-heap 230 MB bereiken op een limiet van 256 MB; de rest van de JVM (metaspace, threads, JIT-code, buffers) overschrijdt de overblijvende 26 MB. Het **totale** geheugen van de cgroup bereikt `memory.max`, de kernel activeert de OOM killer en doodt `java` met `SIGKILL`: code `137`, geen enkele Java-exception (de JVM is nooit zonder heap gevallen, de container is zonder geheugen gevallen). Bevestiging: `podman inspect api --format '{{.State.OOMKilled}}'` → `true`, en `podman events --filter event=oom`. Correcties: `MaxRAMPercentage` verlagen naar 50-75 %, of `--memory` verhogen.

Waarom. De cgroup-limiet geldt voor **heel** het proces; de heap is daar maar een deel van. `OutOfMemoryError` treedt alleen op als de **heap** vol is, wat nooit gebeurt wanneer de kernel eerder toeslaat.

Nuance. `-XX:+ExitOnOutOfMemoryError` en een `--restart` verbergen het symptoom zonder het te verhelpen. Dimensioneren is meten: `podman stats` tijdens een belastingstest.

Voorbeeld.

```
podman inspect api --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'   # oom=true exit=137
podman exec api sh -c 'java -XX:MaxRAMPercentage=90 -XX:+PrintFlagsFinal -version 2>/dev/null | grep " MaxHeapSize"'
```

Vraag 3 — `stats`, `top`, `inspect`

Antwoord. `podman stats` leest de **cgroup-tellers** in real time: CPU %, gebruikt geheugen / limiet, netwerk, I/O. `podman top` somt de **processen** van de container op (UID, PID, commando), zoals `ps` van binnenuit. `podman inspect` beschrijft de **configuratie en de toestand**: wat gevraagd is (limieten, variabelen, mounts), niet wat verbruikt wordt. Zonder `--memory` heeft de cgroup geen plafond (`memory.max = max`) en toont `stats` het geheugen **van de machine** als limiet: 32 GB betekent niet "gereserveerd", maar "niets houdt deze container eerder tegen".

Waarom. Drie bronnen: de kernel (cgroups, `/proc`) voor `stats` en `top`, de database van Podman voor `inspect`.

Nuance. `stats` in rootless vereist cgroup-delegatie; `top` aanvaardt kolommen (`podman top api user pid args`) maar niet de volledige syntaxis van `ps`.

Voorbeeld.

```
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
podman inspect api --format 'mem={{.HostConfig.Memory}}'   # 0 = geen limiet
```

Vraag 4 — De verdwenen `HEALTHCHECK`

Antwoord. Buildah bouwt standaard een image in het **OCI**-formaat, waarvan de specificatie geen veld voor de healthcheck heeft: de instructie wordt genegeerd, met één enkele waarschuwing die verloren gaat in de builduitvoer (`HEALTHCHECK is not supported for OCI image format and will be ignored`). Controle: `podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'` → `<nil>`. Correctie: `podman build --format docker -t api:1.1 .`, waarna dezelfde inspect `[CMD-SHELL wget …]` toont en `podman ps` `(healthy)` weergeeft. Docker bouwt standaard in het Docker-formaat: daar blijft de healthcheck altijd bewaard.

Waarom. `HEALTHCHECK` is een Docker-uitbreiding van het imageformaat, nooit overgenomen in de OCI-norm (die die rol aan de orchestrator laat).

Nuance. Een OCI-image blijft het juiste doel voor Kubernetes, dat de `HEALTHCHECK` hoe dan ook negeert en zijn eigen probes gebruikt. Het docker-formaat telt alleen voor Podman/Docker/Compose.

Voorbeeld.

```
podman build --format docker -q -t api:1.1 . && podman image inspect api:1.1 --format
'{{.Config.Healthcheck.Test}}'
```

Vraag 5 — `unhealthy`, en dan?

Antwoord. Met Podman alleen: **niets**. De toestand wordt `unhealthy` in `podman ps`, er wordt een gebeurtenis `health_status` uitgezonden, de container blijft draaien en slecht antwoorden. Docker doet hetzelfde: de healthcheck is een **indicator**, geen hersteller; alleen Swarm of Compose met `depends_on` gebruiken hem. Podman voegt `--health-on-failure=kill|restart|stop` toe: met `kill` en `--restart=always` wordt de container gedood en herstart. De orchestrator blijft het echte antwoord omdat hij de probe combineert met het **wegnemen van het verkeer** (readiness), de **vervanging** (liveness) en het **verplaatsen** als de machine de oorzaak is.

Waarom. Beslissen om een dienst te doden is een sturingsbeslissing, die de containerengine bewust aan een hogere laag overlaat.

Nuance. Een slecht afgestelde liveness probe doodt een trage dienst in een lus: de probe moet de **levendigheid** testen, niet de belasting.

Voorbeeld.

```
podman run -d --name api --health-on-failure=kill --restart=always api:1.1
podman events --stream=false --since 10m --filter event=health_status | tail -2
```

Vraag 6 — De verharde container

Antwoord. `uid=1000(…) gid=1000(…)` (de image heeft `USER 1000:1000`, `--user 1000` bevestigt het); `touch: /app/x: Read-only file system`; `touch: /tmp/x: Read-only file system` **ook**, want er is geen `--tmpfs /tmp` meegegeven; `java -version` werkt (lezen vraagt niets). Voor een echte Spring Boot-API: `--tmpfs /tmp` toevoegen (Tomcat maakt er zijn werkmap, de JVM schrijft er `hsperfdata`), en als de applicatie bestanden schrijft (uploads, cache), een eigen volume. Verder niets: een API heeft geen enkele capability nodig.

Waarom. `--read-only` vergrendelt de hele image, `/tmp` inbegrepen; legitieme schrijffacties moeten een expliciete mount hebben.

Nuance. `--cap-drop ALL` met `--user 1000` is bijna overbodig (een niet-root heeft de capabilities al niet), maar beschermt tegen een setuid-binary in de image — wat `no-new-privileges` ook vergrendelt.

Voorbeeld.

```
podman run --rm --read-only --tmpfs /tmp --cap-drop ALL --user 1000 --security-opt no-new-
privileges --entrypoint sh api:1.0 -c 'touch /tmp/x && echo OK'
```

Vraag 7 — Root zonder macht

Antwoord. In het eerste geval beschikt root over de standaardcapabilities van de container, waaronder `CAP_CHOWN`; `chown` slaagt. Met `--cap-drop ALL` is het proces nog altijd UID 0, maar zonder **enige** capability: de kernel weigert `chown` (`Operation not permitted`), zoals hij het een gewone gebruiker zou weigeren. Het voorbeeld toont dat "root" onder Linux maar een nummer is; de **privileges** zijn capabilities die aan het proces hangen en die je een voor een kunt wegnemen. Een aanvaller die root wordt in een container met `--cap-drop ALL` heeft maar de rechten van een gebruiker zonder macht op een bestandssysteem dat hij niet kan wijzigen.

Waarom. Sinds Linux 2.2 toetsen de privilegecontroles van de kernel capabilities, niet het UID; UID 0 is maar één manier om de volledige set te krijgen.

Nuance. In rootless gelden de capabilities van de container alleen in de **user namespace**: zelfs `CAP_SYS_ADMIN` geeft er geen enkele macht over de host. Twee beschermingslagen.

Voorbeeld.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo OK'          # OK
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'      #
Operation not permitted
```

Vraag 8 — `latest` in productie

Antwoord. Drie incidenten: (1) **twee servers, twee versies** — een `pull` om 10.00 uur, een andere om 10.05 uur na een nieuwe build, en niemand weet wat waar draait; (2) **terugdraaien onmogelijk** — `latest` is overschreven, de vorige image heeft geen naam meer; (3) **niet-reproduceerbare uitrol** — een `pull` in acceptatie en een in productie geven niet dezelfde inhoud, de test bewijst niets. Strategie: `api:1.4.2` + `api:1.4` + `api:1` + `api:git-<sha>`, nooit `latest` uitrollen. Uitrollen **via digest** garandeert bovendien dat zelfs een **herschreven** versietag (per ongeluk, of door een aanvaller van de registry) niet verandert wat draait: de digest is de vingerafdruk van de inhoud, een tag is maar een naam.

Waarom. Een tag is een verplaatsbare wijzer opgeslagen aan registrykant; een digest wordt berekend uit de lagen.

Nuance. Glijdende tags (`1.4`, `1`) zijn nuttig voor **consumenten** (een basisimage), niet voor de uitrol van **jouw** applicatie.

Voorbeeld.

```
podman image inspect api:1.0 --format '{{.Digest}}'
podman pull localhost:5000/api@sha256:52b82ed9…   # onveranderlijk, wat er ook met de tag gebeurt
```

Vraag 9 — HTTPS verwacht

Antwoord. Podman spreekt standaard **TLS** met elke registry, ook `localhost:5000`; `registry:2` zonder certificaat antwoordt in klare HTTP, vandaar `server gave HTTP response to HTTPS client`. Lokale test: `--tls-verify=false` bij `push` en `pull`. Bedrijf: een geldig **certificaat** op de registry (en de CA in `/etc/containers/certs.d/<registry>/ca.crt` als die intern is). Daartussen declareert `[[registry]] location="…" insecure=true` in `registries.conf` de uitzondering één keer voor altijd. De eerste mag nooit de tweede worden omdat een registry in klare tekst zich laat **onderscheppen en vervangen**: een image die onderweg op het netwerk vervangen wordt, compromitteert de hele uitrolketen.

Waarom. De registry verspreidt uitvoerbare code; TLS garandeert de herkomst en de integriteit onderweg.

Nuance. Docker weigert op dezelfde manier en wordt ingesteld via `insecure-registries` in `daemon.json` — een instelling van de daemon, dus van de beheerder.

Voorbeeld.

```
podman push --tls-verify=false localhost:5000/api:1.0 && curl -s localhost:5000/v2/_catalog
```

Vraag 10 — Hetzelfde `IMAGE ID` na de heen-en-terugreis

Antwoord. De registry heeft de **lagen** (blobs), de **configuratie** en het **manifest** van de image ontvangen, elk geïdentificeerd door zijn hash; bij de `pull` heeft Podman elke hash gecontroleerd en exact dezelfde image opnieuw samengesteld. Het `IMAGE ID` is de hash van de configuratie: identieke inhoud, identiek ID — de naam `localhost:5000/api:1.0` is maar een extra tag op dezelfde lokale image. `--format '{{.Digest}}'` geeft de hash van het **manifest**, die de registry gebruikt: daarnaar verwijs je voor een onveranderlijke uitrol, en hij verandert als een laag verandert.

Waarom. De opslag is **inhoudsgeadresseerd**: twee wegen naar dezelfde inhoud leiden naar hetzelfde object.

Nuance. Dezelfde inhoud gepusht naar twee registry's kan twee manifestdigests hebben als het formaat verschilt (OCI tegenover Docker); het configuratie-ID blijft identiek.

Voorbeeld.

```
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep api     # twee namen, één ID
curl -s localhost:5000/v2/api/manifests/1.0 -H 'Accept:
application/vnd.oci.image.manifest.v1+json' | head -c 200
```

Vraag 11 — `system prune -a --volumes -f` midden in de CI

Antwoord. Verwijderd: alle **gestopte** containers, de ongebruikte netwerken, **alle** images zonder draaiende container (`-a`: basisimages, `api:1.0`, buildcache), en alle volumes die geen enkele **bestaande** container gebruikt. Niet verwijderd: de draaiende containers, hun images en hun volumes — de teststack overleeft… tot haar `down`, waarna haar volumes verweesd zijn. Volgende build: basisimages opnieuw downloaden (minuten, Docker Hub-quotum), herbouwen zonder cache, en verlies van de datavolumes die tussen jobs gedeeld werden. Juiste reeks: `podman system df`, `podman image prune` (alleen dangling), `podman container prune` als de gestopte containers bekend zijn, **benoemde** verwijdering van volumes na lezing van `volume ls --filter dangling=true`.

Waarom. `prune` redeneert in "gerefereerd door een bestaande container", niet in "morgen nuttig".

Nuance. Op een kortstondige CI-agent (bij elke job opnieuw aangemaakt) is `prune -a` juist de norm; het probleem zit bij de **blijvende** agent waar de cache kostbaar is.

Voorbeeld.

```
podman system df && podman volume ls --filter dangling=true && podman image prune -f
```

Vraag 12 — De Quadlet

Antwoord. `daemon-reload` laat de Quadlet-generator het bestand `api.container` lezen, die een unit `api.service` produceert met een `ExecStart=podman run … localhost/api:1.0` en de vertaalde opties (`-p 18080:8080`, `--env-file`). `enable --now` start de container en schrijft hem in bij `default.target`: bij elke start van de gebruikerssessie herstart systemd `podman run`. `--restart=always` faalt na `wsl --shutdown` omdat geen enkel Podman-proces overleeft om het toe te passen; hier is het **systemd** dat door het systeem herstart wordt, en dat Podman herstart. Op WSL heb je `loginctl enable-linger $USER` nodig opdat de `--user`-sessie bestaat zonder aanmelding, en moet de distributie gestart zijn (een open `wsl` of een geplande taak).

Waarom. Een systemd-dienst heeft een permanente toezichthouder; een container met `--restart` heeft er geen zonder daemon.

Nuance. Quadlet genereert **wegwerpcontainers** (`podman run --rm --replace`): de gegevens moeten in volumes zitten, idealiter gedeclareerd met een `.volume`-Quadlet.

Voorbeeld.

```
systemctl --user daemon-reload && systemctl --user enable --now api && systemctl --user status api --no-pager | head -3
journalctl --user -u api -n 5 --no-pager
```

Vraag 13 — De pipeline

Antwoord. (1) **Build** — `podman build --format docker -t api:1.4.2 -t api:git-a1b2c3d .` (labs 04-05); mislukking als de Dockerfile of de tests van de stage `test` falen. (2) **Scan** — Trivy/Grype op de image (lab 05); mislukking bij een kritieke CVE. (3) **Push** — `podman push` naar de privéregistry, met TLS (labs 02, 10); mislukking als de registry weigert (rechten, quotum). (4) **Acceptatie** — `podman compose -f compose.yaml -f compose.staging.yaml up -d` op de image **via digest**, integratietests, `down -v` (lab 09); mislukking als een test of een healthcheck niet slaagt. (5) **Productie** — de orchestrator rolt **dezelfde digest** uit (labs 08, 11+), met de limieten, de `securityContext` en de probes in de manifesten. De scan komt vóór de push (je verspreidt geen kwetsbare image); de `HEALTHCHECK` zit in de image vanaf de build; de resourcelimieten staan in de uitrolbestanden (Compose of manifesten), niet in de image; de digest wordt bij de build berekend, bij de push geregistreerd, en vanaf acceptatie gebruikt.

Waarom. Elke stap levert een controleerbaar en onveranderlijk artefact voor de volgende: image, scanrapport, digest, testresultaat.

Nuance. De pipeline is dezelfde met Docker; alleen de build/push-commando's en het imageformaat verschillen.

Voorbeeld.

```
DIGEST=$(podman image inspect api:1.4.2 --format '{{.Digest}}') && echo "deploy: registry.example.com/api@$DIGEST"
```